

Ammar Boualem Zeghough

Individual Development Roadmap

Respondent 49 | Current role: Graphic Designer

Target direction: Software development, cybersecurity and applied AI

Preferred learning mode: Instructor-led learning

This public-safe roadmap excludes private contact and sensitive personal information from the source assessment.

Ammar Boualem Zeghough - Individual Development Roadmap

Respondent: 49

Current role: Graphic Designer

Target direction: Software development, cybersecurity and applied AI

Preferred learning mode: Instructor-led learning

Current learning allocation: 6-10 hours/week

Additional capacity: 11-15 hours/week

Development objective

The roadmap converts Ammar's existing creative, managerial and practical IT strengths into a disciplined technical progression. The first objective is not to specialise too early. It is to build a reliable base in programming, web systems, Linux, networking, Git and security before choosing a deeper specialisation.

Success definition

By the end of 12 months, Ammar should have a public-safe technical portfolio containing working software, documented labs, security evidence and at least one applied AI project. Progress is measured by deliverables, code quality, problem-solving ability and the ability to explain technical decisions.

Core tracks

1. Software engineering: Python, JavaScript, Git, APIs, testing and web applications.
2. Cybersecurity: networking, Linux, identity, defensive security, secure coding and incident fundamentals.
3. Applied AI: Python data tooling, model/API integration, prompt/system design, evaluation and responsible deployment.
4. Professional capability: technical communication, documentation, teamwork, leadership and portfolio presentation.

Capability profile

Existing strengths

- Graphic design and visual communication.
- Broad practical IT aptitude and a strong self-described ability to learn quickly.
- Hands-on interest in repairing and developing electronic devices.
- Previous management experience in a private firm working across advertising printing, secretarial services and computer work.
- Comfortable working independently and with others.
- Flexible work preference.
- Interest in teaching and creating learning content.

Development needs

- Programming fundamentals and structured problem solving.
- Software engineering workflow: Git, testing, debugging and code review.
- Web application architecture and API integration.
- Cybersecurity foundations and practical defensive labs.
- AI programming and safe model integration.
- A portfolio that demonstrates employable capability rather than only certificates.

Public profiles

- [GitHub](<https://github.com/zeghoughdz-ship-it>)
- [LinkedIn](<https://www.linkedin.com/in/ammam-boualem-zeghough-89981a219/>)
- [Design portfolio](<https://www.tasmeemme.com/zeghoughomario>)

This public IDR intentionally excludes private contact details and sensitive personal information supplied in the assessment.

Career pathway

Recommended pathway

Graphic Designer -> Junior Technical Builder -> Software / Security Generalist -> Specialist

The pathway deliberately preserves Ammar's design capability as an advantage. Front-end engineering, secure product design, developer experience, technical documentation and AI-assisted product development all benefit from strong visual communication.

Gate 1 - Technical foundations

Demonstrate Git fluency, command-line comfort, basic Python and JavaScript, networking concepts, and a deployed static/web project.

Gate 2 - Application builder

Build a small full-stack application with authentication, an API, tests and deployment documentation.

Gate 3 - Security-capable developer

Perform threat modelling, secure the application, run defensive labs and document findings using responsible disclosure practices.

Gate 4 - Applied AI builder

Integrate an AI service or local model workflow into a useful application with evaluation, guardrails and clear limitations.

Gate 5 - Specialisation decision

Choose the strongest evidence-backed direction: software engineering, application security / SOC, or applied AI engineering.

12-month roadmap

Phase	Months	Focus	Required evidence
Foundation	1-2	Python, JavaScript, Git, Linux, networking	GitHub exercises, CLI journal, mini web app
Build	3-4	Web apps, APIs, databases, testing	Full-stack project v1
Secure	5-6	Security fundamentals, secure coding, defensive labs	Threat model, hardening report, security lab notes
Automate	7-8	Python automation, APIs, CI/CD, containers	Automation toolkit + pipeline
Apply AI	9-10	AI APIs/models, evaluation, RAG concepts, safety	AI-enabled application
Specialise	11-12	Chosen track + capstone	Capstone, case study, interview-ready portfolio

Month 1-2: foundation

Complete Python and JavaScript fundamentals, Git/GitHub workflow, Linux command line, networking basics and HTTP. Build small programs weekly.

Month 3-4: build

Create a database-backed web application. Learn API design, authentication concepts, input validation, logging and test fundamentals.

Month 5-6: secure

Study security principles, IAM, OWASP concepts, endpoint/network telemetry and incident basics. Harden the application created in the previous phase.

Month 7-8: automate

Automate repetitive workflows with Python. Introduce containers and CI/CD. Add automated tests and dependency checks.

Month 9-10: apply AI

Learn safe AI integration, prompt/system design, evaluation, retrieval concepts and privacy boundaries. Add a well-scoped AI capability to a real project.

Month 11-12: specialise

Select a primary track based on demonstrated performance and motivation. Produce a polished capstone and prepare for role interviews or internal opportunities.

First 12-week applied sprint

Week	Objective	Practical deliverable
1	Developer environment and Git	GitHub repo, README, branching exercise
2	Python fundamentals	CLI utility with functions and error handling
3	HTML/CSS/JavaScript	Responsive portfolio page
4	Networking + HTTP	Network/HTTP notes and request inspection lab
5	Linux fundamentals	Linux command workbook and shell tasks
6	Web APIs	JavaScript or Python API client
7	Database fundamentals	CRUD app with SQLite/PostgreSQL
8	Secure coding	Input validation, secrets handling and dependency review
9	Cybersecurity lab	Defensive lab report with safe sample telemetry
10	Testing + CI	Automated test suite and CI workflow
11	AI programming	Small AI-assisted feature with evaluation cases
12	Capstone review	Demo, technical write-up and next-quarter decision

Weekly rhythm

- Instructor-led session: 2-4 hours.
- Guided lab: 4-6 hours.
- Independent build: 6-10 hours.
- Review/documentation: 2-3 hours.
- Optional stretch work: only after core evidence is complete.

Sprint gates

Week 4: explain Git, HTTP, basic networking and JavaScript fundamentals.

Week 8: demonstrate a working data-backed application and basic secure coding.

Week 12: present a capstone with code, documentation, tests and a reflective review.

Curriculum architecture

Module A - Programming foundations

Python syntax, data structures, functions, modules, exceptions, files, testing and debugging.

Module B - Web engineering

HTML, CSS, JavaScript, DOM, HTTP, REST APIs, Node or Python backend fundamentals, database access and deployment.

Module C - Systems and networking

Linux, processes, permissions, TCP/IP, DNS, HTTP/TLS, common ports, basic cloud concepts and logging.

Module D - Cybersecurity

CIA triad, identity and access, threat modelling, OWASP risks, endpoint/network telemetry, secure configuration, incident fundamentals and responsible lab practice.

Module E - Automation and DevOps

Git workflows, CI/CD, containers, environment configuration, dependency management, automated tests and release hygiene.

Module F - Applied AI

Python data handling, model/API integration, prompt/system instructions, structured outputs, retrieval concepts, evaluation, privacy and model limitations.

Module G - Professional capability

Technical writing, demos, stakeholder communication, peer review, estimation, teamwork and evidence-based self-assessment.

Practical labs

1. Git lab: branching, pull request simulation, merge conflict resolution.
2. Python lab: parse a CSV/JSON dataset and produce a useful summary.
3. Web lab: build a responsive page using semantic HTML and accessible forms.
4. HTTP lab: inspect requests, responses, headers, status codes and TLS basics.
5. Linux lab: users, permissions, services, logs and shell navigation.
6. API lab: consume an external API and handle failures safely.
7. Database lab: schema design, CRUD operations and parameterised queries.
8. Secure coding lab: validation, secrets, dependency review and error handling.
9. Defensive security lab: analyse safe sample logs and produce an incident note.
10. CI lab: run tests and linting automatically on each change.
11. AI lab: implement an AI feature and evaluate it against fixed test cases.
12. Capstone lab: integrate software, security and AI evidence into one project.

All labs must use authorised systems, local environments, intentionally vulnerable training targets, or synthetic data.

Assignments

Assignment 1 - Technical baseline

Create a skills matrix and identify evidence already available versus capability that still needs to be demonstrated.

Assignment 2 - Explain a system

Write a two-page technical explanation of how a browser reaches a web application, covering DNS, TCP/IP, TLS and HTTP.

Assignment 3 - Build and review

Create a small application, request peer review and implement the feedback.

Assignment 4 - Threat model

Produce a lightweight threat model for the application: assets, trust boundaries, threats, controls and residual risks.

Assignment 5 - AI evaluation

Create 15-20 test cases for an AI feature and document accuracy, failure modes and mitigation.

Assignment 6 - Career evidence review

Map portfolio evidence to target-role requirements and identify the next two highest-value gaps.

Portfolio projects

Project 1 - Secure creative portfolio CMS

A small content-management application for design work with authentication, role separation, file handling and an audit log.

Evidence: architecture diagram, code, tests, security checklist and deployment guide.

Project 2 - Security event triage dashboard

A safe local application that ingests synthetic security events and supports filtering, severity classification and analyst notes.

Evidence: data model, parsing code, detection logic, screenshots and incident-analysis examples.

Project 3 - AI-assisted design brief analyser

An application that converts a design brief into structured requirements, risks, questions and an implementation checklist.

Evidence: prompt/system design, schema, evaluation set, limitations, privacy notes and demo.

Capstone - Secure AI-enabled web application

Combine application development, authentication, logging, secure configuration, automated tests and a constrained AI feature.

The capstone should be deployable, documented and explainable in an interview.

Credentials strategy

Certifications should support demonstrated capability rather than replace it.

Foundation candidates

- Entry-level programming or web development credential where useful.
- Networking fundamentals.
- Linux fundamentals.
- Introductory cybersecurity credential.

Progression candidates

Choose credentials only after practical evidence identifies the strongest pathway:

- Software: cloud developer / DevOps fundamentals.
- Cybersecurity: Security+, vendor security fundamentals or an equivalent defensive-security pathway.
- AI: cloud AI fundamentals followed by role-relevant applied AI certification.

Rule

No more than one major certification objective per quarter unless the required project evidence is already on track.

Study calendar

Ammar currently allocates 6-10 hours per week and indicated capacity for 11-15 additional hours. The roadmap therefore uses a sustainable target of 17-20 hours/week, with a maximum stretch ceiling of 25 hours.

Day	Activity	Target	
Monday	Instructor-led learning / guided theory	2h	
Tuesday	Coding practice	2-3h	
Wednesday	Lab / project work	3h	
Thursday	Coding + security practice	2-3h	
Friday	Review / documentation	1-2h	
Saturday	Deep project block	5-6h	
Sunday	Rest or optional catch-up	0-2h	

Review cadence

- Weekly self-review.
- Fortnightly mentor or coach check-in.
- Formal review at weeks 4, 8 and 12.
- Quarterly roadmap revision after the sprint.

Development KPIs

KPI	12-week target	12-month target	
Public-safe GitHub repositories	3	8+	
Completed practical labs	10	35+	
Portfolio-grade projects	1	4	
Automated tests on active projects	1 project	all major projects	
Technical write-ups	4	12+	
Peer/mentor reviews incorporated	3	10+	
Demonstrations/presentations	2	6+	
Certifications	optional 1	2-3 evidence-aligned credentials	

Quality gates

A deliverable counts only when it is:

1. reproducible,
2. documented,
3. safe to share,
4. reviewed or self-tested,
5. understandable to someone other than the author.

Mentor and review framework

Ammar requested mentoring, especially around programming, cybersecurity and AI programming.

Mentor profile

The best mentor is someone who can review code, explain engineering trade-offs and connect security principles to practical application development.

Cadence

- Fortnightly: 30-minute progress review.
- Monthly: code/project review.
- Quarterly: roadmap and specialisation decision.

Review questions

- What did you build?
- What failed, and what did you learn?
- What evidence proves the new capability?
- Which concept can you now explain without notes?
- What is the highest-value gap for the next two weeks?

Governance and privacy

This IDR is designed for a public or semi-public learning environment.

Never publish

- Personal phone numbers or private email addresses.
- Banking, payment or identity information.
- Passwords, tokens, API keys or private certificates.
- Employer/client confidential information.
- Production logs containing personal or customer data.
- Unauthorised vulnerability details or exploit instructions targeting real systems.
- Private tenant IDs, internal URLs or sensitive administrative screenshots.

Security practice

Use local labs, synthetic data, authorised environments and intentionally vulnerable training targets. Security work must remain defensive, educational and permission-based.

Learning resources

Programming

- Python documentation and beginner tutorials.
- MDN Web Docs for HTML, CSS and JavaScript.
- Git and GitHub documentation.

Systems

- Linux command-line documentation.
- Networking fundamentals resources from reputable vendor academies.

Cybersecurity

- OWASP Top 10 and OWASP Cheat Sheet Series.
- MITRE ATT&CK; for defensive mapping.
- Vendor-neutral security fundamentals.

AI

- Python data tooling.
- Official documentation for the AI platform selected for project work.
- Evaluation-first tutorials covering structured outputs, testing and privacy.

Learning method

Because Ammar prefers instructor-led learning, resources should support a taught programme rather than replace instruction. Each resource should connect to a lab or deliverable.

Download and review

Downloadable PDF is included with the website files.

Review process

1. Read the roadmap and 12-week sprint.
2. Confirm the first-quarter technical focus with the assigned mentor or manager.
3. Adjust weekly hours if the plan is not sustainable.
4. Select instructor-led courses that map directly to the curriculum.
5. Review evidence at weeks 4, 8 and 12.
6. Update the next quarter based on demonstrated capability.